

Rapport de Laboratoire : Analyse et Attaque sur ECDSA

Auteur

- Ouweis Harun

Introduction

Dans ce laboratoire, nous analysons des failles potentielles dans l'implémentation d'ECDSA et réalisons des attaques spécifiques, notamment à l'aide de la réduction de base (LLL). Nous explorons comment récupérer une clé privée dans des contextes où les nonces ou les algorithmes déterministes présentent des faiblesses.

1. Questions Théoriques

1.1. Pourquoi le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est-il une combinaison linéaire des lignes de la matrice M ?

Réponse :

Le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est une combinaison linéaire des lignes de la matrice M , car la matrice M est construite pour encoder les contraintes linéaires imposées par le problème du nombre caché. Voici pourquoi :

1. Matrice M et système d'équations linéaires

1. La matrice M est construite à partir des équations linéaires qui relient les signatures (r, s) , les messages m , et la clé privée α . Chaque ligne de la matrice représente une de ces contraintes :

$$t_i \cdot \alpha + a_i \equiv b_i \pmod{p}$$

où $t_i = r_i/s_i$ et $a_i = -H(m_i)/s_i$.

2. Cette relation est exprimée dans M par :

- Les coefficients t_i et a_i apparaissent dans la dernière colonne et l'avant-dernière colonne de la matrice.
- Les termes b_i , qui représentent les décalages modulaires, sont liés aux entrées principales de M .

2. Structure de la matrice M

La matrice M est définie comme suit :

$$M = \begin{bmatrix} p & 0 & \cdots & 0 & t_1 & a_1 \\ 0 & p & \cdots & 0 & t_2 & a_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & p & t_m & a_m \\ 0 & 0 & \cdots & 0 & B/p & 0 \\ 0 & 0 & \cdots & 0 & 0 & B \end{bmatrix}$$

3. Combinaison linéaire

Pour que $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ soit une combinaison linéaire des lignes de M , cela signifie qu'il existe des coefficients scalaires $\lambda_1, \lambda_2, \dots, \lambda_{m+2}$ tels que :

$$\lambda_1 \cdot \text{ligne}_1 + \lambda_2 \cdot \text{ligne}_2 + \dots + \lambda_{m+2} \cdot \text{ligne}_{m+2} = (b_1, b_2, \dots, b_m, -B\alpha/p, B)$$

Chaque composante du vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est obtenue par une combinaison des éléments correspondants des lignes de M , en tenant compte des scalaires liés aux valeurs b_i , α , et B .

4. Justification par construction

- Les termes $b_1, b_2, \dots, b_m$ correspondent aux composantes générées par les b_i dans les équations linéaires associées.
- Le terme $-B\alpha/p$ résulte de la combinaison des contraintes impliquant $t_i \cdot \alpha$ (les colonnes t_i) et les coefficients de normalisation B/p dans la matrice.
- Le terme B est directement aligné sur la dernière ligne de M , ce qui garantit sa cohérence.

Conclusion

Le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est une combinaison linéaire des lignes de M , car M est spécifiquement construite pour capturer les relations linéaires du problème du nombre caché, où chaque ligne encode une équation ou une contrainte du système.

1.2. Pourquoi le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est-il petit comparé à p ?

Réponse :

Le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est considéré comme "petit" comparé à p car ses composantes ont été définies dans un espace où leur amplitude est limitée par des contraintes liées à la taille de B , α , et b_i . Voici les détails :

1. Petitesse des b_i

1. Définition des b_i :

- Dans le problème du nombre caché, les b_i sont définis comme :

$$b_i = t_i \cdot \alpha - a_i \mod p$$

où t_i et a_i sont dérivés des signatures et des messages.

2. Petitesse intrinsèque :

- Les b_i sont contraints par B , une borne choisie explicitement pour être petite relative à p . Typiquement $B \ll p$, donc chaque $b_i < B$, ce qui en limite la taille.

2. Petitesse de $-B\alpha/p$

1. Rôle du facteur B/p :

- Le coefficient $-B\alpha/p$ est obtenu en normalisant α par B/p . Cette normalisation réduit considérablement la contribution de α , puisque $B \ll p$.
- Même si α est potentiellement grand (puisque'il est modulo n), le facteur B/p garantit que la valeur résultante est beaucoup plus petite que p .

2. Scalabilité contrôlée :

- L'utilisation explicite de B/p dans la matrice M est conçue pour compresser l'échelle des coefficients liés à α , afin de garantir que le vecteur reste "court".

3. Petitesse de B

1. Position du B dans le vecteur :

- Le terme B est directement issu de la dernière ligne de la matrice M , où B est choisi comme une petite constante.

2. Comparaison avec p :

- Par définition, $B \ll p$. Par exemple, si $B = p/2^{32}$, alors B est environ 2^{32} fois plus petit que p .

4. Impact global sur la taille du vecteur

1. Amplitudes des composantes :

- Les $b_1, b_2, \dots, b_m$ sont tous bornés par B , donc leur taille maximale est B .
- Les termes $-B\alpha/p$ et B sont également limités par B , grâce aux facteurs de normalisation.

2. Vecteur court :

- Le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est construit spécifiquement pour être "court", avec chaque composante ayant une magnitude bien inférieure à p .
- Ce design garantit que ce vecteur peut être identifié comme une solution courte par la réduction de base via l'algorithme LLL.

1.3. Que fait l'algorithme LLL ?

Réponse :

L'algorithme LLL (Lenstra–Lenstra–Lovász) est une méthode de réduction de base dans des réseaux (lattices). Il vise à transformer une base donnée en une base réduite composée de vecteurs "courts" et "presque orthogonaux". Voici son fonctionnement et son rôle dans le contexte du problème :

1. Objectif principal

- L'objectif de l'algorithme LLL est de prendre une matrice définissant un réseau (lattice) et de produire une base réduite où les vecteurs sont plus courts et mieux conditionnés pour résoudre des problèmes.

2. Fonctionnement de LLL

1. Réduction de la base :

- LLL applique une procédure itérative pour réduire la longueur des vecteurs de la base tout en conservant leur indépendance linéaire.

2. Propriétés des vecteurs réduits :

- Les vecteurs obtenus après réduction sont plus courts (c'est-à-dire leur norme est plus petite) que les vecteurs initiaux.
- Ils sont également presque orthogonaux, ce qui facilite leur manipulation dans des algorithmes ultérieurs.

3. Application dans le problème

Dans le contexte du problème du nombre caché :

1. Transformation en un réseau :

- La matrice M encode les relations linéaires des signatures et inclut des contraintes liées à la petite taille de α et des b_i .
- Ces contraintes transforment le problème en un réseau où la solution $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$ est un vecteur court.

2. Réduction avec LLL :

- LLL est utilisé pour réduire la matrice M , en produisant une base contenant des vecteurs courts.
- Le vecteur $(b_1, b_2, \dots, b_m, -B\alpha/p, B)$, représentant la clé privée α , est identifié comme l'un des vecteurs courts de cette base.

3. Extraction de la solution :

- La solution α est extraite directement d'un vecteur court produit par LLL, car ce vecteur respecte les contraintes définies dans M .

2. Challenge 1

2.1. Conversion du problème en un problème du nombre caché

L'objectif ici est de convertir le problème de récupération de la clé privée α de l'ECDSA, avec des informations partielles sur le nonce k , en un problème du nombre caché (**Hidden Number Problem**, HNP).

Problème de base dans ECDSA

Dans ECDSA, chaque signature est définie par une paire (r, s) calculée comme suit :

1. $r = (k \cdot G)_x \pmod n$, où G est un point de la courbe elliptique, et k est le nonce aléatoire.
2. $s = k^{-1} \cdot (H(m) + a \cdot r) \pmod n$, où :
 - $H(m)$ est le hash du message m ,
 - a est la clé privée,
 - k est un entier aléatoire.

Faiblesse exploitée

Si une partie du nonce k est connue ou contrainte, par exemple lorsque les τ bits les plus significatifs de k sont 0, cela réduit considérablement l'espace de recherche de k . Cela rend possible la récupération de a en reformulant le problème comme suit :

- La relation entre r , s , $H(m)$, et k implique :

$$k \cdot s - (H(m) + r \cdot a) \equiv 0 \pmod n$$

- En isolant k et a , on obtient une relation linéaire qui peut être exprimée sous forme matricielle.

Reformulation en problème HNP

En introduisant les notations $t_i = r_i/s_i$ et $a_i = -H(m_i)/s_i$ pour chaque signature i , la relation ci-dessus devient :

$$t_i \cdot \alpha - a_i \equiv b_i \pmod n, \quad \text{où } b_i \text{ est une petite erreur due à la faiblesse de } k.$$

Cela correspond au problème du nombre caché (HNP), où :

- $\alpha = a$ (la clé privée) est le secret à trouver,
- t_i et a_i sont calculés à partir des signatures,
- b_i est contraint par la taille réduite du nonce k .

En résumé, le problème ECDSA est converti en un problème linéaire à résoudre modulo n , ce qui permet d'utiliser des outils comme la réduction de base (LLL) pour trouver des solutions efficaces.

2.2. Récupération de la clé privée

Une fois le problème converti en un problème HNP, j'utilise une matrice pour représenter les équations et appliquer une réduction de base afin de récupérer la clé privée α .

Construction de la matrice

La matrice est construite de manière à représenter les contraintes du problème HNP. Voici les étapes :

1. Taille de la matrice :

- La matrice M est de taille $(m + 2) \times (m + 2)$, où m est le nombre de signatures (ici $m = 20$).

2. Remplissage des lignes :

- Les t_i sont placés sur la dernière ligne avant la diagonale,
- Les a_i sont placés sur la dernière ligne,
- Les termes B/p et B sont ajoutés dans la diagonale.

Réduction de base avec LLL

L'algorithme LLL est utilisé pour trouver des vecteurs courts dans l'espace défini par la matrice. Cela permet d'obtenir des solutions proches du problème original. Les vecteurs courts retournés par LLL contiennent la clé privée α .

Extraction et vérification de α

1. Parmi les vecteurs courts retournés, on identifie celui qui respecte la contrainte $v[-1] = B$.
2. La clé privée est calculée comme :

$$\alpha = (-v[-2] \cdot n/B) \mod n$$

3. La clé privée α est validée en recalculant la clé publique $A = \alpha \cdot G$ et en comparant avec la clé publique fournie.

En appliquant cela je parviens à récupérer la clé privée pour le challenge 1.

Challenge 1 : Attaque par réduction de base

Clé privée trouvée :

237833619022848696639005404355272004209706566700873127236229246158457058933444093
25463703570090965103157116327473602

3. Challenge 2

3.1. Explication du problème de sign2

La fonction `sign2` implémente une version déterministe de l'ECDSA, où le nonce k , normalement aléatoire, est calculé de manière prévisible. Cette implémentation est vulnérable car la sécurité de

l'ECDSA repose sur un nonce k imprévisible. Voici comment fonctionne `sign2` et pourquoi cela pose problème :

1. Génération du nonce k :

- k est calculé en utilisant le hash SHA-256 de m comme clé pour un chiffrement ChaCha20 avec un nonce fixe (24 octets de zéros).
- Cette méthode rend k complètement déterministe et prévisible pour tout attaquant connaissant m .

Puisque cette méthode est entièrement déterministe, un même message m produit toujours le même nonce k .

2. Impact de la prévisibilité de k :

- Étant donné que k peut être recalculé à partir de m , un attaquant ayant accès au message m et à la signature correspondante (r, s) peut résoudre l'équation fondamentale d'ECDSA pour récupérer la clé privée a :

$$s \cdot k \equiv H(m) + r \cdot a \pmod{n}$$

- Cela permet d'isoler a :

$$a \equiv (s \cdot k - H(m)) \cdot r^{-1} \pmod{n}$$

- Puisque k est prévisible, la sécurité de la signature est compromise.

3.2. L'attaque sur `sign2`

Pour casser `sign2`, j'ai suivi les étapes suivantes :

1. Recalcul de k :

- En utilisant la même méthode que `sign2`, j'ai dérivé k pour chaque message m . Cela exploite la prévisibilité de k , car :
 - k est calculé à partir du hash de m via ChaCha20 avec un nonce constant.
 - Une fonction `derive_nonce` reproduit exactement cette logique.

2. Calcul de la clé privée a :

- Une fois k connu, j'ai utilisé l'équation d'ECDSA pour isoler a :

$$a \equiv (s \cdot k - H(m)) \cdot r^{-1} \pmod{n}$$

- Cela utilise r , s , et $H(m)$ provenant des signatures et des messages donnés.

3. Validation de a :

- La clé privée a obtenue a été validée en recalculant la clé publique $A = a \cdot G$ et en comparant avec la clé publique donnée.

En appliquant cela je réusis à récupérer la clé privée pour le challenge 2.

Challenge 2 : Attaque sur sign2

Clé privée trouvée :

820682598933094467007983093230781755859680299797213088595406947066147898645298712
3193146492729051282841800617203021

4. Challenge 3

4.1. Explication du problème de sign3

La fonction `sign3` implémente une version déterministe de l'ECDSA, où le nonce k , normalement aléatoire, est calculé de manière fixe à partir de la clé privée a . Cela introduit une vulnérabilité importante, car le même nonce k est utilisé pour toutes les signatures générées avec la même clé privée.

1. Génération déterministe du nonce k :

- k est dérivé en utilisant la clé privée a comme suit :
 - Un hash SHA-256 de a sert de clé pour ChaCha20.
 - Le nonce pour ChaCha20 est également dérivé du hash SHA-256 de a (les 24 premiers octets).
 - ChaCha20 produit un flux chiffré à partir duquel k est extrait.
- Puisque a est fixe, le nonce k est identique pour toutes les signatures.

2. Impact sur la sécurité :

- Le même k entraîne des signatures avec le même r , puisque $r = (k \cdot G)_x$.
- Cela crée une relation linéaire entre les signatures (r, s) associées à deux messages distincts m_1 et m_2 . En combinant ces signatures, un attaquant peut isoler k , puis calculer a .

4.2. Attaque sur sign3

Pour exploiter la vulnérabilité, j'ai suivi les étapes suivantes :

1. Identification des signatures utilisant le même nonce k :

- J'ai parcouru les signatures pour trouver celles ayant un r identique. Cela indique que le même nonce k a été utilisé pour ces signatures.
- Une fois une paire de signatures avec le même r identifiée, j'ai utilisé leurs équations pour calculer k .

2. Calcul de k :

- Les deux signatures (r_1, s_1) et (r_2, s_2) avec le même k satisfont les équations :

$$s_1 \cdot k - H(m_1) \equiv r \cdot a \pmod{n}$$

$$s_2 \cdot k - H(m_2) \equiv r \cdot a \pmod{n}$$

- En soustrayant les deux, j'élimine a et isole k :

$$(s_1 - s_2) \cdot k \equiv H(m_1) - H(m_2) \pmod{n}$$

- Le nonce k est calculé comme :

$$k = \frac{H(m_1) - H(m_2)}{s_1 - s_2} \pmod{n}$$

3. Calcul de la clé privée a :

- Une fois k connu, a est calculé à partir de l'équation d'ECDSA pour s_1 :

$$a = \frac{s_1 \cdot k - H(m_1)}{r} \pmod{n}$$

4. Validation de a :

- La clé publique A a été recalculée comme $A = a \cdot G$ et comparée à la clé publique fournie. Cela a permis de vérifier la validité de a .

En appliquant cela je finis par récupérer la clé privée pour le challenge 3.

Challenge 3 : Attaque sur sign3

Signatures utilisant le même nonce trouvées : 0 et 1

Nonce k trouvé :

328128853134713759466786855151559324284110303591007632007316355404734032830355880
05291271749278926013423213860589819

Clé privée trouvée :

360501192232114648412411773098428472334892702465060332018757401094776609309432451
87618777520429470779784923577543647

Clé privée validée :

360501192232114648412411773098428472334892702465060332018757401094776609309432451
87618777520429470779784923577543647

5. Challenge 4

5.1. Explication du problème de sign4

La fonction `sign4` implémente une version déterministe de l'ECDSA où le nonce k est calculé comme suit :

$$k = \text{int}(\text{SHA256}(\text{str}(a) + \text{str}(m)))$$

Cela introduit une dépendance explicite entre k et la clé privée a , ainsi que le message m .

Problème de sécurité

1. Génération déterministe de k :

- Chaque message m génère un nonce unique k , mais ce dernier dépend de la clé privée a .
- Cela signifie qu'un attaquant peut modéliser le problème comme un système d'équations linéaires reliant les signatures (r, s) , le nonce k , et la clé privée a .

2. Relation linéaire exploitable :

- Pour chaque signature (r, s) , l'équation fondamentale d'ECDSA est :

$$s \cdot k \equiv H(m) + r \cdot a \pmod{n}$$

- En substituant k avec son expression déterministe, cette équation devient un système linéaire dépendant de a .

3. Approche d'attaque :

- En combinant plusieurs signatures, il est possible de transformer le problème en un **problème du nombre caché**, où a est la valeur inconnue.
- Une réduction de base (LLL) est utilisée pour trouver a en exploitant les relations linéaires des signatures.

5.2. Attaque sur sign4

Pour casser `sign4`, j'ai suivi les étapes suivantes :

1. Transformation en problème du nombre caché

Chaque signature (r, s) a été réécrite sous la forme :

$$t_i \cdot a + a_i \equiv b_i \pmod{n}$$

avec :

- $t_i = \frac{r}{s}$,
- $a_i = \frac{-H(m)}{s}$.

Ces relations linéaires ont été utilisées pour construire une matrice A , encapsulant le problème dans un espace multidimensionnel.

2. Construction de la matrice

- La matrice A est construite pour capturer les contraintes des équations linéaires :
 - Les dimensions sont $(m + 2) \times (m + 2)$, où m est le nombre de signatures.
 - Les coefficients t_i et a_i sont calculés pour chaque signature.
 - Une borne $B = n/2^{32}$ est utilisée pour représenter la petite taille de a .

3. Réduction avec LLL

- L'algorithme LLL est appliqué à A pour obtenir une base réduite contenant des solutions courtes.
- Parmi ces vecteurs, un vecteur contenant a est identifié.

4. Extraction et validation de la clé privée a

- La clé privée a est extraite à partir du vecteur réduit.
- a est validée en recalculant la clé publique $A = a \cdot G$ et en la comparant à A_{public} .

En appliquant cela je réussis à récupérer la clé privée pour le challenge 4.

Challenge 4 : Attaque sur sign4

Clé privée trouvée :

64746123777868181536037317226476367073399866885733819971543235948445344562241876
2017460912123494007679981088467912